Requirements

S2184546

January 28, 2025

Functional		
Requirement	**Description**	**Test approach**
Drones must avoid no-fly zones	Due to safety and privacy concerns, certain areas should be avoided by any drones. It is important that this requirement is followed, even though it may result in less efficient routes. The shape of these no-fly zones could be any polygon.	Use System level tests ensure the safety of the drone path.
Must output route plan in JSON and GeoJSON formats	These file formats are required to verify the correctness and optimality of the routing, as well as making the plans available for programming the drones.	System + Integration
Must accept valid orders	This requires checking that the contents of the order is valid (e.g., the selected pizza is made at the specified restaurant) and that the payment information is valid (payment processing is assumed to be done by an external service, either at the point of order or on completion of the order).	System
The drone must deliver only one order at a time with between 1 and 4 pizzas in each order.	The drone must only deliver one order at a time. This would prevent a mix-up of orders as all of the drones cargo would belong to a single person (the user would only have to identify that the drone is for their order, rather than each of the pizzas). An additional security feature could be that the drones cargo require a passcode that only the user that ordered it would have. This would prevent peoples from taking orders that are not theirs.	Unit tests involving order validation
Must be able to dynamically fetch data from a REST API	For the software to operate in real time, it requires the ability to fetch data over a network.	Integration
Must be able to calculate the distance between 2 points given in longitude and latitude	In order for the drone to navigate, it need to know the distance between itself and different points on the map. Further refinements of the system could add GPS functionality, so that if the drones real position becomes different from its modelled position (where the system thinks the drone is), such as if the drone hits something and is knocked off-course or accuracy errors in its angle, its position could be recalibrated in the system.	Unit

Robustness		
Requirement	**Description**	**Level of Concern**
Data Validation	The software must validate all data retrieved from the REST API (e.g., orders, no-fly zones) to ensure proper formatting and completeness before processing.	Unit
Exception Handling	The system must include exception handling for scenarios such as network failures or invalid data formats, avoiding crashes.	Unit

Security		
Requirement	**Description**	**Level of Concern**
Input Sanitization	All user-provided inputs, including the date and API URL, must be sanitized to prevent injection attacks or malformed data from causing errors.	Unit
Log Sanitization	The system should ensure that the logs contain no sensitive information (credit card information), as that would create a security vulnerability.	System
All interactions with the REST API must use HTTPS	User data, including credit card info, is transmitted from the REST API to the computer running the system. This is extremely important, since unencrypted data can be accessed by eavesdroppers on the network.	Integration

Measurable		
Requirement	**Description**	**Level of Concern**
Must output flight plan in less than 60s	The time spent planning the optimal drone route will add to the total delivery time, meaning that excessive time spent on this is unacceptable. More importantly, the longer the planning takes, the more computational resources it will consume (which could stall the processing of other routes in a scaled up version of this project).	System
The system must process and generate logs for 100% of orders received via the REST API.	Every order must be processed, with it being designated valid or invalid.	Unit + Integration

Qualitative		
Requirement	**Description**	**Level of Concern**
Code Quality	The software must use modular, well-documented code with clear variable names and JavaDoc comments for all public methods and classes. This is particularly important, given that the software will likely need to be continually worked on while the drone delivery service continues to operate.	System
Usability	The application must provide clear console output and error messages for ease of debugging and user interaction.	System
Scalability	The design must accommodate future expansions, such as additional constraints or changes in the REST API structure, with minimal code changes.	System
Compliance	The software must adhere to Java 18 standards and use modern programming constructs such as streams, lambda expressions, and pattern matching where appropriate.	System